

Final Project Book

Orel Dayan 209452093

Ariel University, Department of Computer Science and Mathematics

Advisor: Dr. Erel Segal-Halevi

Abstract

This project extends the Aqua Guardian rehabilitation game with an automated difficulty calibration system for post-stroke patients using the Amadeo robotic device. The system collects patient-specific performance data from five brief calibration trials (~20 seconds each) and trains a personalized Ridge regression model over 10 engineered features to predict oxygen outcomes from game parameters. A three-solver optimization cascade (gradient-based, Monte Carlo, reduced-space) generates parameter configurations for nine target difficulty levels (10%–90% remaining oxygen), enabling therapists to select appropriate challenge levels based on clinical context. Beyond calibration, the project includes code refactoring, bug fixes, and UI improvements (pause controls, parameter persistence, visual feedback). Technical validation using developer self-testing demonstrates the system can learn parameter-difficulty relationships and generate configurations achieving target oxygen levels within the tested parameter space, though clinical validation with real patients remains necessary to assess therapeutic effectiveness. The framework addresses the practical challenge of manual parameter tuning that previously required trial-and-error session restarts, potentially reducing therapist workload and improving session consistency.

Contents

1	Introduction	4
1.1	Solution Overview	4
2	System Architecture	5
2.1	Three-Layer Design	5
2.2	Data Flow Pipeline	5
2.3	Trial System Workflow	6
2.3.1	Trial Execution Flow	6
2.4	Component Interaction Architecture	7
2.5	Data and Configuration Management	8
2.5.1	Configuration Management	8
2.5.2	Data Persistence	9
2.6	User Interface Screenshots	9
3	Game Parameters and Feature Engineering	12
3.1	Parameter Definitions	12
3.2	Parameter Impact on Difficulty	12
3.2.1	Derived Feature (10th feature)	13
3.3	Feature Engineering and Data Model	13
3.4	Input Mode Detection and Feature Adaptation	14
4	Machine Learning Pipeline	15
4.1	Training Process	15
4.2	Regression Model	15
4.2.1	Model Formulation	15
4.2.2	Adaptive Regularization	16
4.2.3	Cross-Validation	16
5	Parameter Optimization and Difficulty Calibration	18
5.1	Optimization Problem Formulation	18
5.2	Optimization Algorithms	18
5.2.1	Primary Solver: Gradient Three-Phase Method	18
5.2.2	Phase 1: Analytical Minimal-Change Solution	19
5.2.3	Phase 2: Projected Gradient Descent Refinement	19
5.2.4	Phase 3: Iterative Best-Solution Tracking	20
5.2.5	Fallback Solvers	20
5.2.6	Solver 2: Biased Random Sweep (Monte Carlo)	20
5.2.7	Solver 3: Reduced Multi-Gradient (Top-Features Fallback)	21
5.2.8	Adaptive Range Constraints	21
5.3	Solver Performance Comparison	22
5.4	Multi-Target Optimization and Output	22
5.5	Python Server Extension	24
5.5.1	Architecture and Workflow	24
5.5.2	Setup Instructions	24
5.5.3	Supported Models	24
5.5.4	Small Dataset Optimization	25

6	Limitations and Challenges	26
7	Conclusion and Future Work	27
8	References	28

1 Introduction

This project extends the Aqua Guardian rehabilitation game, developed in collaboration with Beit Loewenstein Rehabilitation Center for post-stroke finger motor recovery using the Amadeo robotic device. The game enables patients to control an underwater diver navigating caves while collecting oxygen and avoiding obstacles, providing immediate feedback to promote motor practice.

Despite the initial system’s potential, several practical limitations emerged during early deployment at the rehabilitation center. Therapists were required to manually configure nine game parameters through trial-and-error, often needing to restart sessions when parameter selection proved suboptimal. The codebase lacked modularity, contained hardcoded constants, and exhibited unresolved player movement bugs. Additionally, the user interface was missing critical control elements needed for efficient clinical workflows, such as a pause button.

This project addresses two objectives:

1. **System hardening:** Code refactoring, bug fixes, and UI improvements.
2. **Automatic difficulty calibration:** A trial-based system where patients complete five short calibration trials (2 caves each, ~ 20 seconds). Data is used to train patient-specific Ridge regression models over ten features (nine measured parameters plus derived *EffectiveDrainRate*). A three-solver optimization cascade generates parameter configurations for nine target difficulty levels (10%–90% remaining oxygen), enabling therapists to select appropriate challenge levels. Selected parameters are automatically saved to `SelectedParameters.json` and loaded into the main game. An optional Python server supports alternative ML models (ElasticNet, Huber, PLS).

The system maintains backward compatibility with existing gameplay while adding calibration as an optional preprocessing step, ensuring seamless integration into current clinical workflows without disrupting established therapy protocols. All system validation and testing reported in this document was conducted using developer self-testing; clinical validation with actual patients remains necessary to assess therapeutic effectiveness.

1.1 Solution Overview

The system implements a three-stage pipeline:

1. **Trial Collection:** Five calibration trials, each composed of two cave segments. The first segment is fixed across all trials, while the second varies per trial (`cavei.csv`). Trial parameters are initialized from predefined configuration files, depending on the selected mode (fixed or random).
2. **Regression Analysis:** Patient-specific Ridge regression models over 10 engineered features (9 measured + derived *EffectiveDrainRate*).
3. **Multi-Target Optimization:** Three-solver cascade generates parameter configurations for nine target difficulty levels (10%–90% remaining oxygen).

Selected parameters are persisted to JSON and automatically loaded into the main game session.

2 System Architecture

2.1 Three-Layer Design

The system consists of three sequential layers that process gameplay trials and produce personalized difficulty parameters. Each layer completes one stage of the calibration process. Figure 1 provides an overview of the architecture.

1. **Trial Collection Layer:** This layer executes five calibration trials, each consisting of a fixed baseline cave and a second cave loaded from `cavei.csv` ($i = 1, \dots, 5$). It manages trial sequencing, applies game parameters from either fixed or random CSV configurations, handles a single retry if needed, detects the input device (Amadeo robotic device or keyboard), and saves trial results to CSV files. Each trial produces a data point $(\mathbf{x}_i, y_i)$, where $\mathbf{x}_i$ contains nine game parameters and y_i is the terminal oxygen percentage, resulting in a dataset of five training samples $\{(\mathbf{x}_i, y_i)\}_{i=1}^5$, which serves as input to the Regression Analysis Layer.
2. **Regression Analysis Layer:** Receives the trial dataset and processes it through feature extraction, Z-score normalization, and Ridge regression training. The system supports built-in C# Ridge regression (default) or an optional Python server providing alternative models (ElasticNet, Huber, PLS). Both modes train a predictive model $\hat{f}(\mathbf{x})$ from 10 features (9 game parameters plus `EffectiveDrainRate`). Model performance is evaluated using leave-one-out cross-validation. The resulting trained model is passed to the next layer for optimization.
3. **Parameter Selection Layer:** Uses the trained regression model $\hat{f}(\mathbf{x})$ to solve the inverse problem of finding parameter configurations for predefined difficulty targets. It applies a three-solver optimization cascade (gradient-based, random search, multi-gradient descent) to generate parameters for nine target levels (10%–90% oxygen). Results are written to `target.csv` and shown in the UI for therapist selection. The chosen configuration is saved to JSON and automatically loaded at the next session. The optimization approach is detailed in Section 5.1.

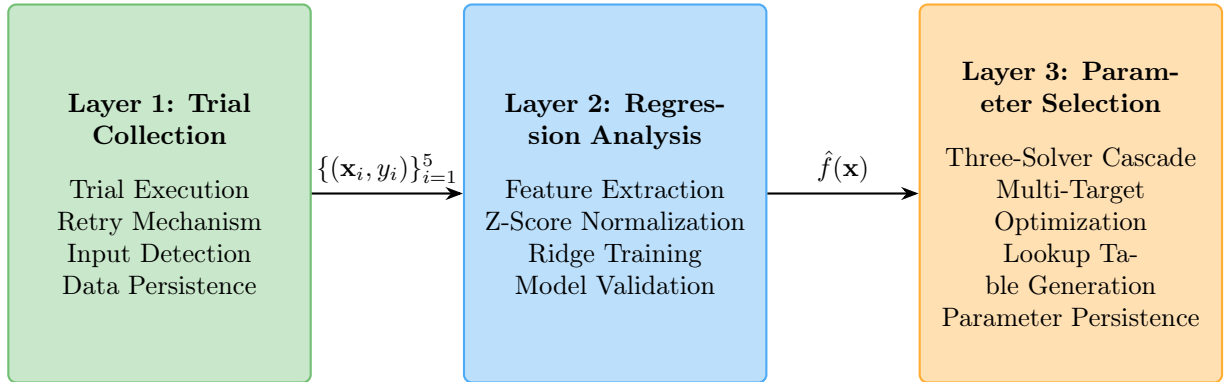


Figure 1: Three-layer system architecture. Layer 1 produces labeled training samples; Layer 2 fits a predictive model $\hat{f}$; Layer 3 inverts the model to find parameters achieving target outcomes.

2.2 Data Flow Pipeline

The calibration process operates as a sequential pipeline that transforms raw trial data into optimized parameter sets, as illustrated in Figure 2. This process directly implements the three-layer architecture described above and visualized in Figure 1.

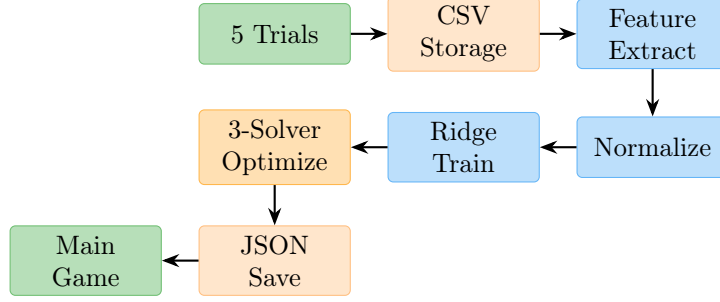


Figure 2: End-to-end calibration pipeline from trials through feature extraction, model training, and optimization to the final gameplay stage

2.3 Trial System Workflow

Layer 1 executes five sequential calibration trials. Each trial uses a cave configuration from `cavei.csv` ($i = 1, \dots, 5$) together with a fixed baseline cave (Figure 3). Each trial produces a data point $(\mathbf{x}_i, y_i)$, where $\mathbf{x}_i$ contains the nine applied game parameters and y_i is the final oxygen percentage achieved by the player.

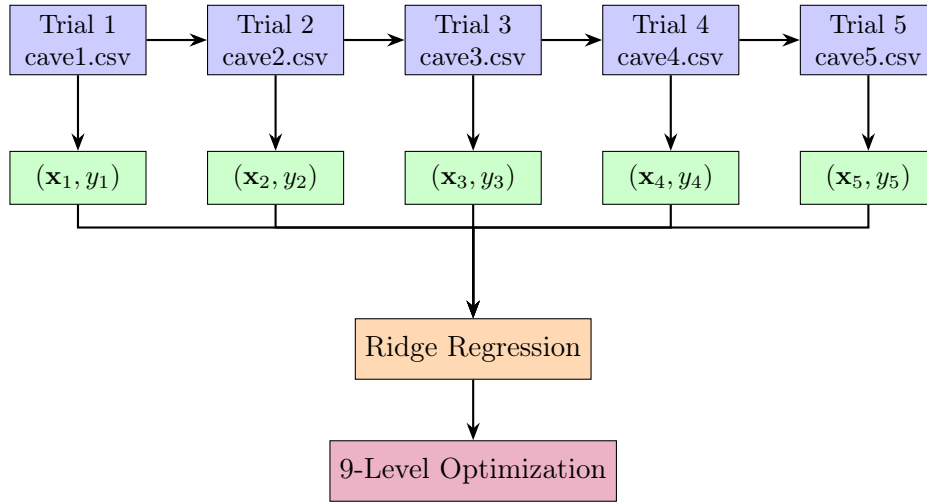


Figure 3: Trial system workflow: 5 sequential trials (each using a different cave configuration) produce 5 training samples $\{(\mathbf{x}_i, y_i)\}$, which are used to train a Ridge regression model and generate 9 optimized difficulty levels.

2.3.1 Trial Execution Flow

Figure 4 shows the complete trial lifecycle including retry handling, providing detailed implementation of the trial workflow shown in Figure 3. The retry mechanism allows up to 2 attempts per trial (original attempt plus one retry). The retry logic operates as follows: (1) If the first attempt fails (oxygen depletes to 0% before completion), the oxygen value is temporarily recorded as 0; (2) If a second attempt succeeds, the stored value is overwritten with the new result; (3) If both attempts fail, 0 is retained. This ensures that only successful trial outcomes or the best retry attempt are recorded, while maintaining data integrity for analysis.

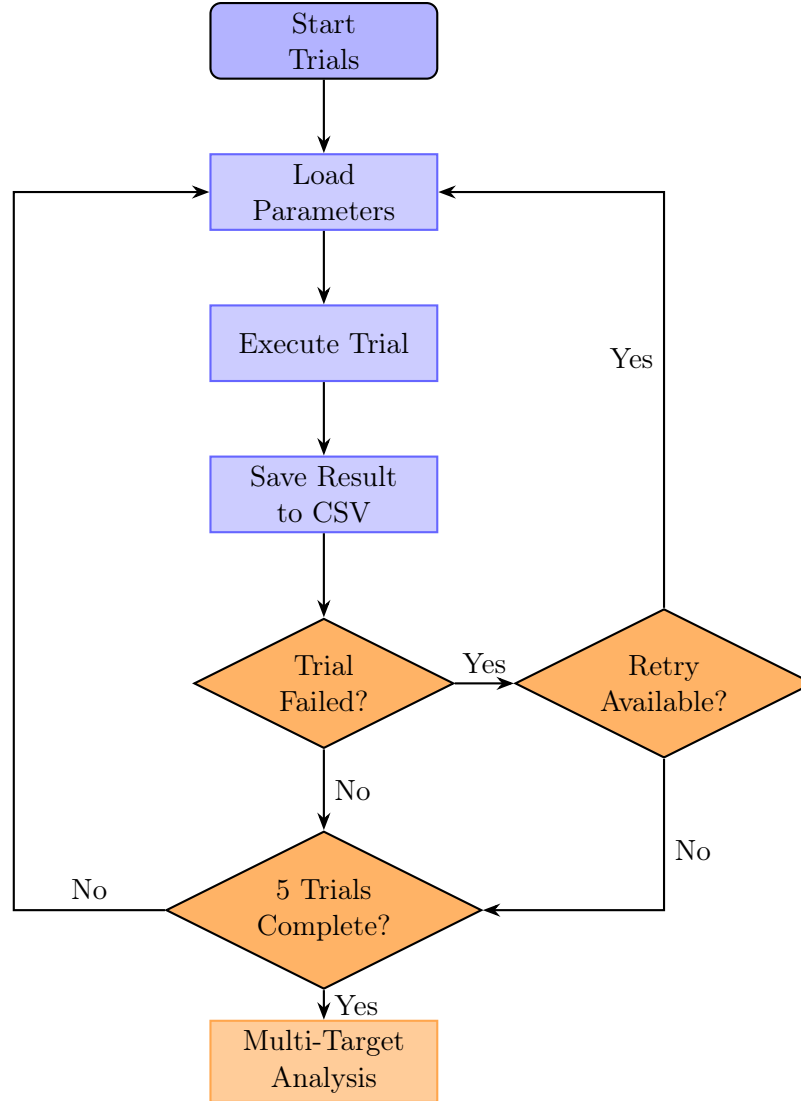


Figure 4: Trial system workflow with retry mechanism

2.4 Component Interaction Architecture

The components described in Section 2 interact through a layered architecture with three main tiers: game controllers (trial execution, cave generation), data persistence, and machine learning modules (feature extraction, regression, optimization). Figure 5 illustrates the detailed data flow and interaction patterns between these layers. Data persistence mechanisms are detailed in Section 2.5.

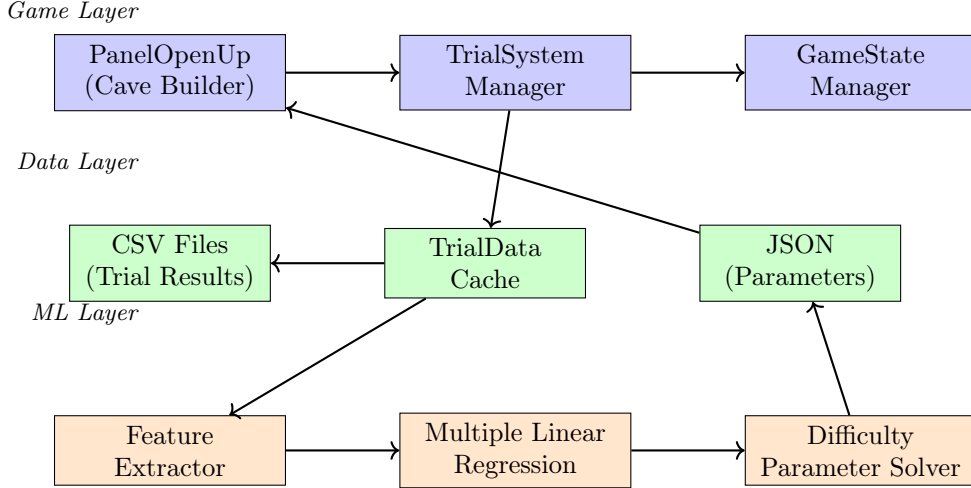


Figure 5: Component interaction diagram illustrating data flow from trial execution through the machine learning pipeline to parameter persistence. Game components (blue) interface with data persistence (green) and ML modules (orange).

2.5 Data and Configuration Management

The system uses centralized configuration management and multiple data persistence mechanisms to support the calibration workflow. Configuration is managed through a Unity ScriptableObject implementing the singleton pattern, which stores system-wide settings. This centralized approach allows easy modification of system behavior without code changes.

Data persistence is handled through three main components: an in-memory cache for temporary data storage during sessions, CSV file persistence for trial results (with dynamic columns to support retry mechanisms), and JSON storage for therapist-selected parameters. The system also tracks game state, managing transitions between calibration and gameplay modes, and monitoring trial completion events.

2.5.1 Configuration Management

The centralized configuration system exposes the following parameter categories:

Category	Parameters
Trial System	Total number of trials (default: 5), trial parameters file path, cave file path pattern, random parameters mode flag
Difficulty Defaults	Oxygen gained per balloon, oxygen drop per second, oxygen drop on collision, interpolation speed factor
Cave Generation	General pivot position, cave placement pivot, arrows-to-wall pivot, chest pivot positions
System Constants	Maximum health (100%), player collision delay, player life wait time, color alpha value

Table 1: Configuration categories in the centralized ScriptableObject system.

2.5.2 Data Persistence

Table 2 summarizes the key data files and their structure:

Table 2: Data files and their structure.	
File	Description
Trial_5_runs_.csv	Fixed calibration parameters; each run appends a new oxygen column (o2_runX).
Trial_Random_Parameters.csv	Randomly sampled parameters; one oxygen result per run (o2_result).
target.csv	9 target levels (10%–90%) with optimized parameters.
SelectedParameters.json	Therapist-selected parameters and metadata.

The system supports two trial protocols: *fixed mode* using `Trial_5_runs_.csv` with predefined parameter configurations and *random mode* using `Trial_Random_Parameters.csv` for random parameter exploration. In random mode, the parameters are generated within a predefined range. The retry logic described in Section 4 applies to both fixed and random modes. On game start, the system automatically loads the most recent therapist-selected configuration from `SelectedParameters.json` into the user interface and game components, ensuring session continuity without recalibration.

2.6 User Interface Screenshots

The system provides several key user interfaces throughout the calibration workflow:



Figure 6: System initialization screen showing game startup and automatic parameter loading from previous sessions, enabling seamless continuation of therapy sessions



Figure 7: Trial start screen displaying current trial parameters and ready state, allowing therapists to review configuration before patient begins the calibration trial

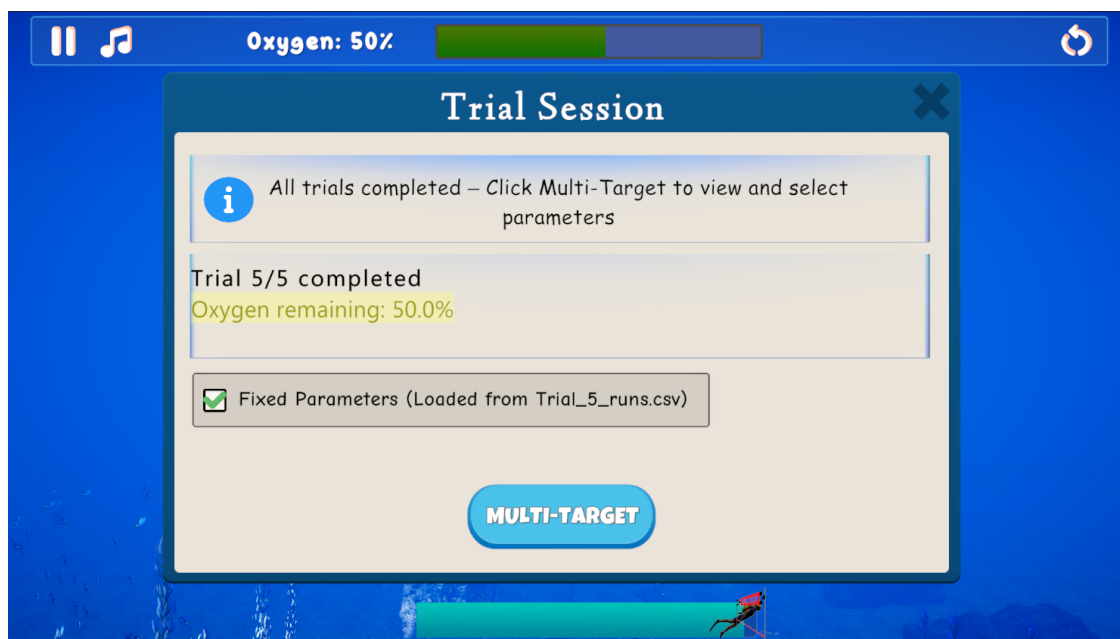


Figure 8: Trial completion screen showing results after all 5 calibration trials have been executed



Figure 9: Interactive difficulty selection table showing 9 target levels with predicted oxygen, error, and optimized parameters for therapist selection

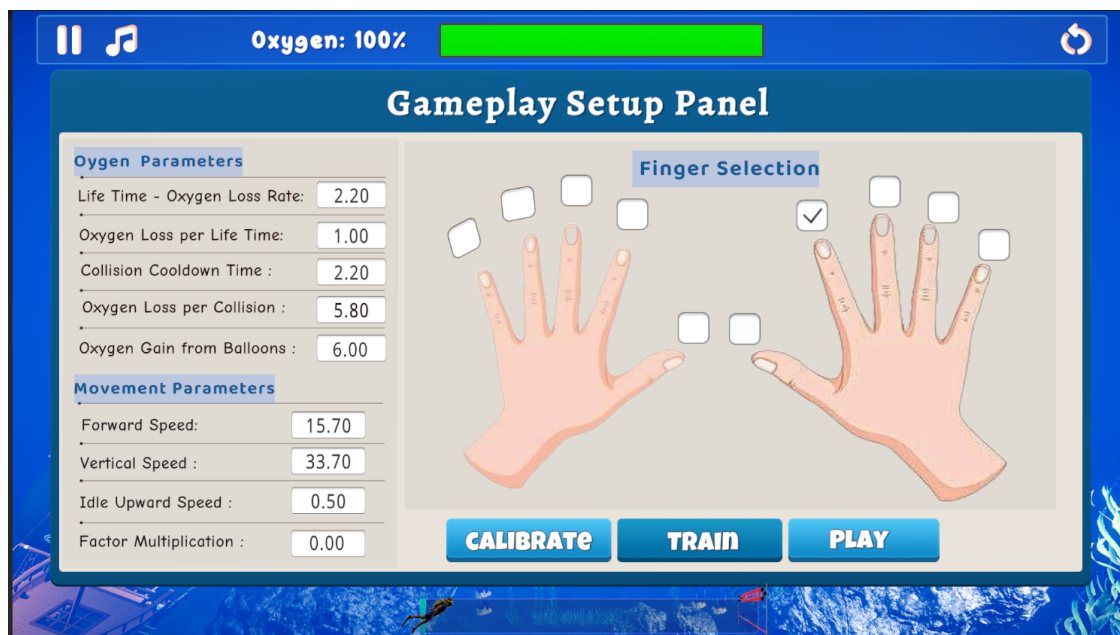


Figure 10: Main gameplay screen after selecting the 90% target difficulty, with the optimized parameters from the chosen row automatically applied to the system

3 Game Parameters and Feature Engineering

The regression model employs 10 features: 9 directly measured game parameters plus one derived feature (**EffectiveDrainRate**). In summary: 9 configurable parameters + 1 derived feature = 10 features + 1 intercept term = 11 total coefficients (β_0 for intercept and $\beta_1, \dots, \beta_{10}$ for features). Feature extraction incorporates input mode adaptations to account for different input devices. All parameters are summarized in Table 3.

3.1 Parameter Definitions

Game difficulty is controlled by nine configurable parameters. Table 3 lists all parameters with their ranges and units.

Table 3: Game parameter ranges.

Parameter	Range	Unit	Game Effect
speed	[10, 40]	units/s	Forward movement speed
verticalSpeed	[15, 45]	units/s	Up/down movement speed
idleUpwardSpeed	[0.01, 8]	units/s	Passive upward drift
lifeTime	[0.5, 4]	seconds	Time interval between oxygen drain cycles
RemoveHealthEveryLifeTime	[1, 7]	% oxygen	Oxygen drain per cycle of life-Time
removeHealthWithCollide	[5, 20]	% oxygen	Collision damage from caves
timeBetweenCollides	[1, 5]	seconds	Collision cooldown between damage events
healHealthPoint	[3, 15]	% oxygen	Oxygen restored per balloon
factorForce	[0.5, 15]	multiplier	Amadeo force sensitivity (keyboard mode: unused)

Note: The ranges shown in Table 3 are default values and can be modified in the Unity editor or code based on therapist clinical judgment.

3.2 Parameter Impact on Difficulty

The following parameters affect game difficulty:

- **speed, verticalSpeed:** Higher speeds increase control difficulty and collision risk
- **idleUpwardSpeed:** Moderate values facilitate balloon collection; excessive values cause ceiling collisions
- **factorForce:** Higher values improve Amadeo control sensitivity (only relevant in Amadeo mode)
- **lifeTime:** Shorter intervals between drain cycles increase difficulty
- **RemoveHealthEveryLifeTime:** Higher values increase oxygen drain rate per cycle

- **removeHealthWithCollide**: Higher values increase collision damage
- **timeBetweenCollides**: Shorter immunity periods increase collision frequency
- **healHealthPoint**: Higher values restore more oxygen per balloon, reducing overall difficulty

3.2.1 Derived Feature (10th feature)

The system computes a derived feature representing oxygen loss per second:

$$\text{EffectiveDrainRate} = \frac{\text{RemoveHealthEveryLifeTime}}{\text{lifeTime}} \quad (1)$$

This feature improves model accuracy by capturing the combined effect of drain rate and timing, allowing the regression model to better predict the impact of game parameters on oxygen depletion. The derived feature makes the instantaneous oxygen loss rate explicit, enabling the model to recognize that different combinations of drain timing and magnitude can produce equivalent difficulty levels.

Motivation for derived feature. The parameters `lifeTime` and `RemoveHealthEveryLifeTime` interact multiplicatively. For example, the pairs (`lifeTime=2.0s`, `RemoveHealthEveryLifeTime=4.0%`) and (`lifeTime=1.0s`, `RemoveHealthEveryLifeTime=2.0%`) both correspond to the same drain rate of 2.0%/s and therefore produce identical oxygen depletion trajectories.

A linear model using only these raw parameters cannot capture this interaction and treats such equivalent settings as different. The derived feature $\text{EffectiveDrainRate} = \frac{\text{RemoveHealthEveryLifeTime}}{\text{lifeTime}}$ makes the instantaneous drain rate explicit and improves predictive accuracy.

Chain rule for optimization. During optimization, `EffectiveDrainRate` is excluded from direct manipulation because infinitely many (`lifeTime`, `RemoveHealthEveryLifeTime`) pairs can produce the same drain rate. Instead, the optimizer adjusts the constituent parameters, with gradient contributions propagated through the chain rule:

$$\beta_{\text{lifeTime}}^{\text{eff}} = \beta_{\text{lifeTime}} - \beta_{\text{EDR}} \cdot \frac{\text{RemoveHealthEveryLifeTime}}{\text{lifeTime}^2} \quad (2)$$

$$\beta_{\text{drain}}^{\text{eff}} = \beta_{\text{drain}} + \beta_{\text{EDR}} \cdot \frac{1}{\text{lifeTime}} \quad (3)$$

where β_{EDR} is the regression coefficient for `EffectiveDrainRate`, and the effective coefficients β^{eff} account for the indirect effect via the derived feature.

3.3 Feature Engineering and Data Model

Each gameplay trial is represented as a 10-dimensional feature vector. Features are indexed 0–9, where:

- **Indices 0–8**: Nine directly configurable game parameters (Table 3)
- **Index 9**: Derived feature $\text{EffectiveDrainRate} = \text{RemoveHealthEveryLifeTime} / \text{lifeTime}$

The derived feature is used for regression analysis to capture the multiplicative interaction between drain timing and magnitude (Section 3.2.1), but excluded from direct optimization to prevent multicollinearity—infinitely many (`lifeTime`, `RemoveHealthEveryLifeTime`) pairs produce

the same drain rate, so the optimizer adjusts these constituent parameters individually with gradient contributions propagated via the chain rule (Equations 2–3). Table 4 summarizes the feature space structure.

Table 4: Feature space structure for regression modeling.

Index	Feature Name	Type
0–8	Nine measured parameters	Measured
9	<code>EffectiveDrainRate = RemoveHealthEveryLifeTime / lifeTime</code>	Derived

3.4 Input Mode Detection and Feature Adaptation

The system automatically detects the input device used during each trial by monitoring actual input events, not just configuration settings. Detection occurs at the end of each trial and evaluates two conditions: (1) whether the system is configured for Amadeo/Emulation mode, and (2) whether Amadeo input was actually used during the trial (as opposed to keyboard fallback). The system only marks a trial as Amadeo mode if both conditions are met, ensuring that trials where the device was configured but not actually used (e.g., due to connection issues) are correctly identified as keyboard mode.

The detected input mode is recorded in the trial data as `IsAmadeoMode` (0 for keyboard, 1 for Amadeo) and is used throughout the analysis pipeline. The feature extraction process automatically adapts features based on detected input mode: in Amadeo mode, `idleUpwardSpeed` is multiplied by 0.5 for improved control, and `factorForce` is used as-is; in keyboard mode, `idleUpwardSpeed` is used unchanged and `factorForce` is set to 0. If no Amadeo trials exist, `factorForce` is automatically excluded from optimization to prevent model instability.

4 Machine Learning Pipeline

The machine learning pipeline (Layer 2 in Section 2) transforms raw calibration data into a predictive model capable of estimating final oxygen percentage from parameter configurations. This section describes the regression model, training procedure, and validation approach.

4.1 Training Process

The training process learns the relationship between game parameters and oxygen outcomes from a small calibration set of $n = 5$ trials. It proceeds in four stages aligned with Figure 11:

1. **Feature extraction:** Raw trial data (9 parameters + oxygen outcome) is augmented with the derived **EffectiveDrainRate** feature (Section 3.2.1), yielding 10 input features per trial.
2. **Z-score normalization:** Each feature is standardized to zero mean and unit variance across the 5 trials. This ensures parameters with different scales (e.g., **speed** in $[10, 40]$ units/s vs. **lifeTime** in $[0.5, 4]$ seconds) contribute equally to the model, preventing large-scale parameters from dominating regression.
3. **Ridge regression training:** The design matrix $\mathbf{X} \in \mathbb{R}^{5 \times 11}$ (5 trials, 10 normalized features + intercept) is constructed. Ridge-regularized normal equations are solved via Cholesky decomposition for numerical stability, yielding 11 coefficients: β_0 (intercept) and $\beta_1, \dots, \beta_{10}$ (feature weights).
4. **Cross-validation evaluation:** Model performance is assessed using leave-one-out cross-validation (LOOCV), computing R^2 , RMSE, and MAE. These metrics estimate predictive accuracy on unseen trials, though with $n = 5$ they likely overestimate true generalization (Section 6).

The trained model yields an intercept β_0 (baseline oxygen level independent of parameters) and 10 feature coefficients $\beta_1, \dots, \beta_{10}$. The sign of β_j indicates whether increasing feature j tends to increase ($\beta_j > 0$, e.g., **healHealthPoint**) or decrease ($\beta_j < 0$, e.g., **RemoveHealthEveryLifeTime**) oxygen, while its magnitude reflects the strength of that effect.

4.2 Regression Model

The system uses Ridge regression to prevent overfitting with small datasets ($n = 5$, $p = 10$) and handle correlated parameters. Given the extremely small sample size ($n = 5$ trials) relative to the number of features ($p = 10$), and the presence of correlated parameters (e.g., **lifeTime** and **RemoveHealthEveryLifeTime**), Ridge regression provides a principled approach to regularization that shrinks coefficients toward zero while maintaining numerical stability. The solution is computed via Cholesky decomposition. The following sections detail the mathematical formulation and training procedure.

4.2.1 Model Formulation

The model processes $n = 5$ calibration trials, each with 10 features. Before training, features are normalized to have zero mean and unit variance using z-score standardization:

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \quad (4)$$

where μ_j and σ_j are the mean and standard deviation of feature j across all trials. This normalization ensures all parameters contribute equally regardless of their original scale.

Ridge regression finds coefficients that minimize prediction error while keeping feature coefficients small:

$$\hat{\boldsymbol{\beta}} = \arg \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \sum_{j=1}^{10} \beta_j^2 \quad (5)$$

where $\mathbf{X} \in \mathbb{R}^{5 \times 11}$ is the design matrix (first column is all ones for the intercept, followed by 10 normalized features), $\mathbf{y} \in \mathbb{R}^5$ contains the observed oxygen levels, $\lambda \geq 0$ controls regularization strength, and $\boldsymbol{\beta} = [\beta_0, \beta_1, \dots, \beta_{10}]^T$ are the model coefficients. The intercept β_0 is not regularized (it represents the baseline oxygen level).

The solution is computed as:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{D})^{-1} \mathbf{X}^T \mathbf{y} \quad (6)$$

where $\mathbf{D} = \text{diag}(0, 1, 1, \dots, 1)$ is a diagonal matrix that applies regularization only to feature coefficients (not the intercept). Once trained, predictions are made using:

$$\hat{O}_2 = \beta_0 + \sum_{j=0}^9 \beta_{j+1} \cdot \tilde{x}_j \quad (7)$$

where $\tilde{x}_0, \dots, \tilde{x}_9$ are the 10 normalized features and $\beta_1, \dots, \beta_{10}$ are their corresponding coefficients.

4.2.2 Adaptive Regularization

The regularization parameter adapts to sample size:

$$\lambda(n) = \text{Clamp}(0.5 + (10 - n) \times 0.2, 0.5, 2.0) \quad (8)$$

For $n = 5$: $\lambda = 1.5$; for $n \geq 10$: $\lambda = 0.5$.¹

4.2.3 Cross-Validation

Model performance is evaluated using Leave-One-Out Cross-Validation (LOOCV):

$$\text{CV-RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_{(-i)})^2} \quad (9)$$

where $\hat{y}_{(-i)}$ is the prediction for trial i using a model trained on the remaining $n - 1$ trials. Metrics computed: R^2 , RMSE, MAE.

¹The formula scales regularization strength inversely with sample size: smaller datasets ($n < 10$) require stronger regularization to prevent overfitting when the number of features ($p = 10$) approaches or exceeds the number of samples. The linear scaling factor (0.2 per sample below 10) ensures a smooth transition, while the bounds (0.5–2.0) prevent extreme values that could either under-regularize (risking overfitting) or over-regularize (causing underfitting). This approach is standard in Ridge regression for small-sample scenarios.

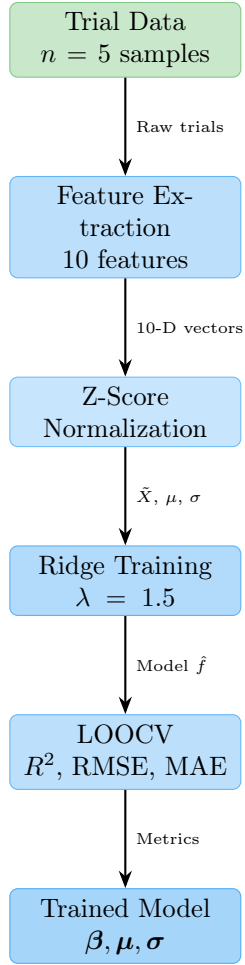


Figure 11: Complete machine learning pipeline showing data transformations from raw calibration trials through feature engineering, normalization, training, and validation to produce a validated regression model with stored normalization parameters.

5 Parameter Optimization and Difficulty Calibration

The system identifies game parameter settings (speed, damage, lifetime, etc.) that produce a target final oxygen level, ranging from 10% (most difficult) to 90% (easiest).

5.1 Optimization Problem Formulation

After training the regression model, the system solves the **inverse problem**: given a target oxygen level, find game parameters that achieve it. This is termed "inverse" because the forward direction predicts oxygen from parameters ($f : \mathbf{x} \rightarrow y$), whereas optimization reverses this relationship to find parameters for a desired oxygen outcome ($f^{-1} : y_{\text{target}} \rightarrow \mathbf{x}^*$). The optimization operates as the final stage of the machine learning pipeline (Section 2), solving the constrained inverse problem for nine target difficulty levels (10%, 20%, ..., 90% oxygen) using a three-solver cascade.

Let $x \in \mathbb{R}^d$ denote the vector of configurable game parameters (speed, verticalSpeed, idleUpwardSpeed, lifeTime, RemoveHealthEveryLifeTime, removeHealthWithCollide, timeBetweenCollides, healHealthPoint, factorForce), and let $f(x)$ denote the predicted final oxygen percentage produced by the trained regression model. The core optimization problem is to find parameter values that minimize the difference between the predicted and target oxygen levels:

$$x^* = \arg \min_x |f(x) - O_2^{\text{target}}| \quad (10)$$

subject to box constraints:

$$x_{\min}^{\text{adapted}} \leq x \leq x_{\max}^{\text{adapted}} \quad (11)$$

where the adapted bounds are derived from observed trial ranges with additional safety buffers, ensuring optimized configurations remain clinically safe and close to the region where the model was trained.

5.2 Optimization Algorithms

The three-solver cascade generates optimized parameters for 9 target oxygen levels (10%–90%) in a single analysis. Each target optimization follows this workflow:

1. Prepares patient baseline from trial history (median/mean per parameter)
2. Determines feature freedom (parameters with adjustment headroom, excluding derived features and fixed mode indicators)
3. Applies adaptive range constraints based on observed data (Section 5.2.8)
4. Executes solver cascade until acceptable solution found (Solver 1 $\rightarrow$ Solver 2 $\rightarrow$ Solver 3)
5. Validates solution against target with error reporting

5.2.1 Primary Solver: Gradient Three-Phase Method

The primary optimization method (**Solver 1**) is a three-phase gradient-based procedure that exploits the linear structure of the Ridge model in normalized feature space and typically achieves prediction errors below 0.5% for interpolated targets (based on technical validation with developer self-testing).

5.2.2 Phase 1: Analytical Minimal-Change Solution

Phase 1 computes a closed-form solution by distributing the required oxygen change across parameters proportionally to their model coefficients. The algorithm proceeds as follows:

1. Initialize from baseline $\mathbf{x}_0$ (mean or median of trial parameters)
2. Predict baseline oxygen: $\hat{y}_0 = \beta_0 + \sum_{j=1}^p \beta_j \tilde{x}_{0j}$ where $\tilde{x}_{0j}$ are normalized features
3. Compute required change: $\Delta y = y_{\text{target}} - \hat{y}_0$
4. Identify free parameters: exclude **EffectiveDrainRate** (derived feature, not directly optimized), fixed mode indicators, and parameters with negligible coefficients ($|\beta_j| < \epsilon$)
5. Distribute change proportionally to coefficient magnitudes:

$$\Delta x_j = \frac{\beta_j^{\text{eff}}}{\sum_{k \in \text{free}} (\beta_k^{\text{eff}})^2} \cdot \Delta y \cdot \sigma_j \quad (12)$$

where σ_j is the feature standard deviation from normalization, and β_j^{eff} are effective coefficients accounting for chain-rule corrections for derived features (Section 3.2.1)

6. Update parameters: $x_j^{(1)} = x_{0j} + \Delta x_j$
7. Clamp to feasible range: $x_j^{(1)} \leftarrow \text{Clamp}(x_j^{(1)}, l_j, u_j)$ where $[l_j, u_j]$ are the adapted bounds

This step produces a fast, stable initial candidate that moves the prediction toward the target with minimal deviation from the patient’s baseline parameter values. The proportional distribution ensures that parameters with larger regression coefficients receive larger adjustments, reflecting their relative importance in affecting oxygen levels.

5.2.3 Phase 2: Projected Gradient Descent Refinement

Phase 2 applies iterative projected gradient descent to refine the Phase 1 solution. The algorithm proceeds as follows:

1. Initialize: $\mathbf{x}^{(0)} \leftarrow$ Phase 1 solution
2. For iteration $t = 1, \dots, T_{\text{max}}$ (typically $T_{\text{max}} = 50$):
 - (a) Predict oxygen level: $\hat{y}^{(t)} = f(\mathbf{x}^{(t-1)}) = \beta_0 + \sum_{j=1}^p \beta_j \tilde{x}_j^{(t-1)}$
 - (b) Compute prediction error: $e^{(t)} = y_{\text{target}} - \hat{y}^{(t)}$
 - (c) If $|e^{(t)}| < \epsilon_{\text{tol}}$ (typically 0.1%), terminate successfully
 - (d) Compute gradient components: for each parameter j , the gradient contribution is $\beta_j^{\text{eff}} \sigma_j$ where σ_j is the normalization standard deviation
 - (e) Adaptive step size: $\alpha^{(t)} = \alpha_0 \cdot \min(1, \frac{|e^{(t)}|}{5})$ where $\alpha_0 = 0.1$ is the base learning rate
 - (f) Update parameters: $x_j^{(t)} = x_j^{(t-1)} + \alpha^{(t)} \cdot \text{sign}(e^{(t)}) \cdot \beta_j^{\text{eff}} \sigma_j$
 - (g) Project to feasible region: $x_j^{(t)} \leftarrow \text{Clamp}(x_j^{(t)}, l_j, u_j)$ where $[l_j, u_j]$ are adapted bounds
3. Return: $\mathbf{x}^{(T)}$ if converged, else Phase 1 solution

The adaptive step size decreases as the error approaches zero, preventing oscillation around the target. For errors $> 5\%$, the algorithm uses the full step size; for errors $< 1\%$, the step size is proportionally reduced. This phase typically converges quickly for targets within or close to the observed oxygen range.

5.2.4 Phase 3: Iterative Best-Solution Tracking

Activated only if Phase 2 error $> 0.5\%$. This phase runs additional iterations ($T_{\max} = 100$) while tracking the best solution encountered:

$$\mathbf{x}_{\text{best}} = \arg \min_{\mathbf{x}^{(t)}} |y_{\text{target}} - f(\mathbf{x}^{(t)})| \quad (13)$$

The algorithm continues gradient descent steps similar to Phase 2, but with more conservative step sizes and extended iteration limits. At each iteration, it evaluates the prediction error $|f(\mathbf{x}^{(t)}) - O_2^{\text{target}}|$ and maintains a record of the parameter vector that achieved the smallest error across all iterations. The loop terminates when no further improvement is observed over a fixed number of iterations or when the maximum iteration count is reached.

The final output of Solver 1 is $\mathbf{x}_{\text{best}}$, not necessarily the last iterate. This best-solution tracking approach improves robustness against occasional overshooting that can occur when gradient steps overshoot the optimal point due to the linear model approximation or constraint boundary effects.

5.2.5 Fallback Solvers

When Solver 1 cannot reach a sufficiently small error (typically for targets requiring strong extrapolation, such as extreme 10% or 90% levels with narrow trial ranges), the system automatically falls back to alternative solvers.

5.2.6 Solver 2: Biased Random Sweep (Monte Carlo)

When Solver 1 produces error $> 5\%$, the system uses Monte Carlo search with biased sampling. The algorithm proceeds as follows:

1. Set random seed: $\text{seed} = \lfloor y_{\text{target}} \times 1000 \rfloor$ (for reproducibility)
2. Generate $N = 300$ candidate configurations:

- (a) Sample $u_j \sim \text{Uniform}(0, 1)$ for each parameter j
- (b) Apply bias transform based on target difficulty:

$$t_j = \begin{cases} u_j^2 & \text{if } y_{\text{target}} < 30\% \text{ (hard targets: bias toward extremes)} \\ u_j & \text{if } 30\% \leq y_{\text{target}} \leq 70\% \text{ (mid-range: uniform)} \\ 1 - (1 - u_j)^2 & \text{if } y_{\text{target}} > 70\% \text{ (easy targets: bias toward extremes)} \end{cases} \quad (14)$$

- (c) Map to parameter range: $x_j = l_j + t_j \cdot (u_j - l_j)$ where $[l_j, u_j]$ are adapted bounds

3. Evaluate all candidates: $\hat{y}_i = f(\mathbf{x}_i)$ for $i = 1, \dots, N$
4. Select best: $\mathbf{x}^* = \arg \min_{\mathbf{x}_i} |y_{\text{target}} - \hat{y}_i|$
5. Return: $\mathbf{x}^*$ with error $|y_{\text{target}} - f(\mathbf{x}^*)|$

The bias rationale: Power transforms (e.g., u_j^2) shift probability density toward parameter extremes for hard/easy targets (10%, 90%) where extrapolation is needed, while maintaining uniform sampling for mid-range targets (40%–60%) where interpolation suffices. Because this method does not rely on gradient information, it remains effective even when the model surface is locally flat or poorly conditioned. A fixed random seed per target oxygen level guarantees deterministic and reproducible results.

5.2.7 Solver 3: Reduced Multi-Gradient (Top-Features Fallback)

Last resort when both Solver 1 and Solver 2 fail. This method reduces optimization to top- $K = 3$ most influential parameters by coefficient magnitude. The algorithm proceeds as follows:

1. Rank parameters by coefficient magnitude: $|\beta_1^{\text{eff}}| \geq |\beta_2^{\text{eff}}| \geq \dots$ after chain-rule adjustments
2. Select top- $K = 3$ parameters (excluding derived features and fixed mode indicators)
3. If < 3 valid parameters exist, use fallback set: `{RemoveHealthEveryLifeTime, removeHealthWithCollide}` (empirically high-impact parameters)
4. Initialize reduced parameter vector $\mathbf{x}_K$ from trial means
5. Fix remaining parameters at trial mean values
6. Run gradient descent on reduced space (similar to Solver 1 Phase 2):
 - (a) For $t = 1, \dots, 100$:
 - (b) Predict: $\hat{y}^{(t)} = f(\mathbf{x}_K^{(t-1)}, \mathbf{x}_{\text{fixed}})$
 - (c) Update only K parameters: $x_j^{(t)} = x_j^{(t-1)} + \alpha \cdot \text{sign}(e) \cdot \beta_j^{\text{eff}} \sigma_j$ where α is a small fixed step size
 - (d) Clamp: $x_j^{(t)} \leftarrow \text{Clamp}(x_j^{(t)}, l_j, u_j)$
 - (e) Track best solution encountered
7. Return: Best configuration found

Design rationale: Reducing from 8–9 parameters to 3 constrains the search space, helping escape local minima. This approach is effective when full gradient descent fails due to conflicting gradients or extreme extrapolation. The fallback parameters (`RemoveHealthEveryLifeTime` and `removeHealthWithCollide`) are empirically high-impact (typically largest $|\beta|$ values), ensuring reasonable solutions even with poor models. Even when the model is weak or the target lies far outside the calibration range, Solver 3 guarantees a valid solution within bounds, albeit sometimes with a larger residual error than Solver 1 or Solver 2.

5.2.8 Adaptive Range Constraints

All three solvers operate within adaptive parameter bounds that constrain the search space. These bounds are computed based on observed trial data to improve extrapolation performance while ensuring safety. Search ranges are computed as observed parameter bounds plus an adaptive buffer b that scales with target distance $\Delta_{\text{target}} = |O_{\text{mean}} - O_{\text{target}}|$:

$$[l_j, u_j] = [\max(l_{\text{global}}, x_{\min}^{\text{obs}} - b), \min(u_{\text{global}}, x_{\max}^{\text{obs}} + b)] \quad (15)$$

Buffer scaling: $b = 10\%$ for $|\Delta y| < 20\%$, linear interpolation (10%–50%) for $20\% \leq |\Delta y| < 40\%$, and full global bounds for $|\Delta y| \geq 40\%$. Two modes are supported: **Conservative** (10%–25% buffer) for interpolation, and **Expanded** (25%–50% buffer) for extreme targets.

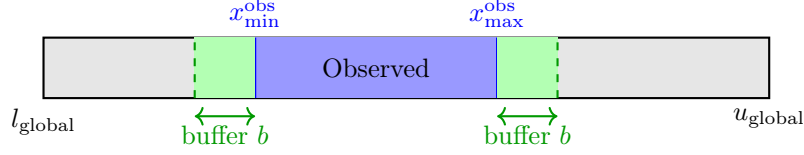


Figure 12: Adaptive range constraints: observed range (blue) plus buffer (green) within global bounds (gray).

5.3 Solver Performance Comparison

Table 5 compares the performance characteristics of the three solvers.

Table 5: Optimization solver comparison.

Solver	Type	Best For	Typical Error
Gradient 3-Phase	Analytical + Gradient	Dense coefficients (Ridge)	$< 0.5\%$
RandomSweep	Monte Carlo	Universal fallback	$< 2\%$
Multi-Gradient	Gradient descent	Last resort	Varies

5.4 Multi-Target Optimization and Output

The calibration framework generates a full difficulty spectrum for nine target oxygen levels (10%–90%) rather than optimizing for a single level. This approach addresses domain-transfer challenges: calibration trials are shorter than full gameplay (2 vs. 21 caves), and patient behavior changes due to fatigue and learning. By generating multiple difficulty levels, therapists can choose the appropriate challenge level for each patient. The three-solver cascade is applied independently to each target, yielding optimized parameter vectors and estimated errors.

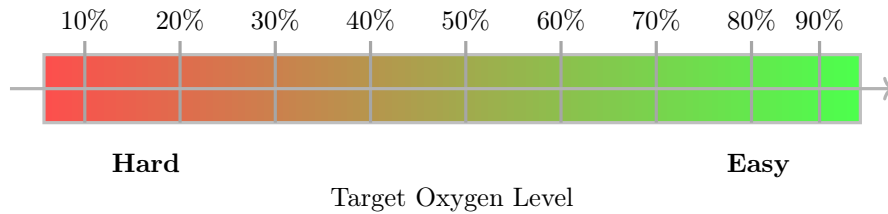


Figure 13: Multi-target difficulty spectrum from 10% (hard) to 90% (easy). System generates optimized parameters for all 9 levels in a single analysis.

Results are stored in two files:

- **target.csv**: Lookup table with 9 rows (one per difficulty level) containing **oxygenTarget**, **predicted_oxygen**, **error**, and all nine parameter values, used by therapists to select difficulty.

- **MultiTarget_Report_.csv**: Detailed report including all lookup table parameters plus meta-data (timestamp, model R^2 , RMSE, input type, number of trials, average error, solver method per target).

Results are presented in the Regression UI as a lookup table. The therapist selects a difficulty level based on clinical context; the chosen configuration is persisted to **SelectedParameters.json** and automatically loaded by the main game in subsequent sessions.

Table 6: MultiTarget_Report_.csv structure (excerpt showing 3 of 9 targets).

Target	Predicted	Error	Speed	vSpeed	lifeTime	Drain	Heal
10%	10.23%	0.23%	38.5	42.1	0.86	5.37	6.55
30%	29.90%	0.10%	29.8	35.2	1.50	5.60	6.00
50%	49.57%	0.43%	19.1	29.9	1.14	3.23	6.05
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
90%	89.95%	0.05%	15.3	22.5	3.80	1.20	14.8

Report Metadata: Timestamp: 2025-12-16 14:35:22, Model R^2 : 0.847, RMSE: 4.23%, Input: Keyboard, Trials: 5, Average Error: 0.21%

Note: These metrics reflect technical validation using developer self-testing, not clinical validation with patients.

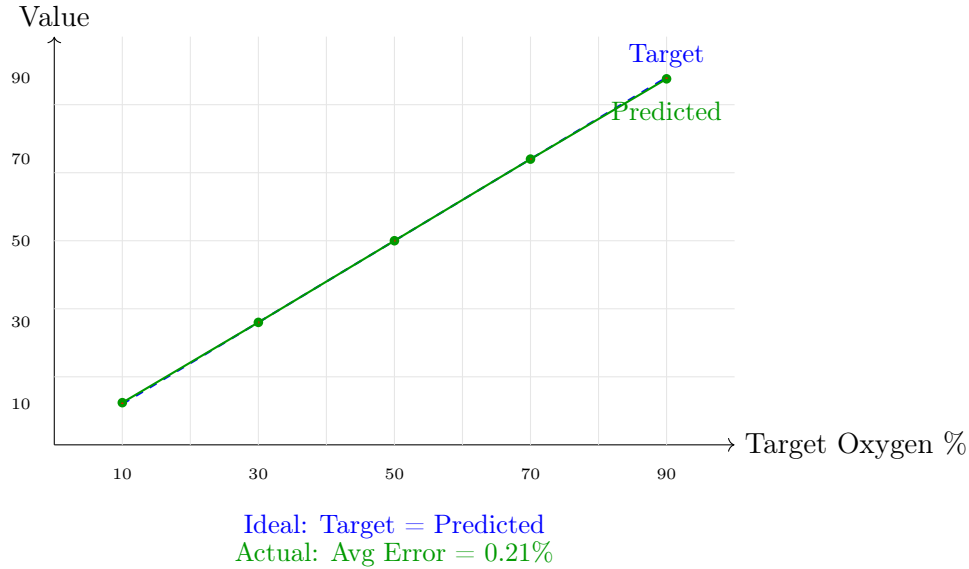


Figure 14: Multi-target optimization results: predicted oxygen closely matches targets across the full difficulty spectrum. Results shown are from technical validation using developer self-testing.

5.5 Python Server Extension

Besides the built-in Unity C# Ridge regression, the system also supports an optional **Python server**. This server uses Python’s advanced machine learning libraries to improve accuracy and allow performance comparisons with Unity’s model.

Because it needs an external server and is less reliable for deployment, the Python version is used only for testing and research. In clinical or offline use, the system automatically switches back to the standard Unity version if the Python server is not available.

5.5.1 Architecture and Workflow

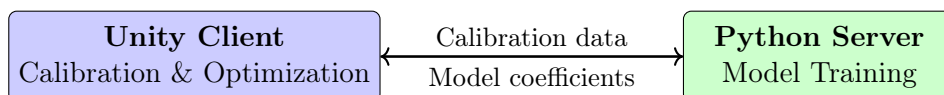


Figure 15: Unity–Python architecture: Unity sends calibration data for model training and receives coefficients for parameter optimization.

The workflow proceeds as follows:

1. Unity sends calibration trial data (9 parameters + oxygen outcomes) to the Python server via HTTP/JSON.
2. The Python server trains the selected regression model and returns model coefficients $(\beta_0, \beta_1, \dots, \beta_{10})$ and normalization parameters.
3. Unity uses these coefficients for the three-solver optimization cascade to generate parameter configurations for nine target difficulty levels.

5.5.2 Setup Instructions

1. Install Python packages: `pip install numpy pandas scikit-learn flask flask-cors`
2. Start the Python server: `cd PythonScripts && python regression_server.py`
3. In Unity: Enable `PythonRegressionServerClient` and choose the desired model.

5.5.3 Supported Models

The Python server provides four regression model options (Table 7): Ridge (default, matching Unity’s implementation), ElasticNet (sparse solutions with automatic feature selection), Huber (robust to outliers), and Partial Least Squares (PLS, effective for correlated features). Model selection is configured in Unity through the `PythonRegressionServerClient` component.

Table 7: Regression models supported by the Python server.

Model	Main Advantage
Ridge	Default model (same as Unity)
ElasticNet	Sparse solutions with feature selection
Huber	Robust to outliers and noisy data
PLS	Effective for correlated input features

5.5.4 Small Dataset Optimization

For datasets with five trials, the `/train_small` endpoint applies Lasso-based feature selection (Algorithm 1) to automatically identify the most influential features, reducing the risk of overfitting. The server uses Leave-One-Out cross-validation to estimate model performance:

$$\text{CV-RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_{(-i)})^2}$$

where $\hat{y}_{(-i)}$ is the prediction for sample i when the model is trained on all other samples.

Algorithm 1 Lasso Feature Selection for Small Datasets

- 1: **Input:** Features X , targets y , desired count K
 - 2: Standardize X with z-score normalization
 - 3: **for** $\alpha \in \{10, 5, 2, 1, 0.5, 0.2, 0.1, 0.05\}$ **do**
 - 4: Train Lasso model with regularization α
 - 5: Count non-zero coefficients n_{selected}
 - 6: **if** $n_{\text{selected}} = K$ **then**
 - 7: **break**
 - 8: **end if**
 - 9: **end for**
 - 10: **Output:** indices of top- K features by $|\beta_j|$
-

6 Limitations and Challenges

This section summarizes the main limitations of the current system and the technical challenges encountered during its development, along with the measures taken to address them.

1. **Small sample size:** With only five calibration trials, the regression model operates in a highly underdetermined regime. Ridge regularization and Leave-One-Out cross-validation mitigate overfitting, but predictive accuracy remains limited.
2. **Extrapolation uncertainty:** Extreme difficulty targets (10%, 90%) require extrapolation beyond observed data. A three-solver cascade with adaptive constraints manages this risk.
3. **Model linearity assumptions:** Linear relationships may miss complex parameter interactions. Larger datasets will enable nonlinear models in future iterations.
4. **Inconsistent input modes:** Mixed Amadeo/keyboard sessions cause feature inconsistencies. Automatic input mode detection ensures consistent data handling.
5. **Limited clinical validation:** System validation focused on technical performance metrics (developer self-testing, R^2 , RMSE, optimization error) rather than clinical outcomes. Technical validation demonstrates the system can learn parameter-difficulty relationships, but patient-based validation and therapist feedback remain essential to assess therapeutic effectiveness and appropriateness for real patients.
6. **Design complexity:** The multi-solver optimization cascade may appear over-engineered for the small dataset size. This design prioritizes robustness and safety over model simplicity, ensuring reliable parameter generation even with limited calibration data.
7. **Codebase complexity:** The existing codebase contained hardcoded values, mixed responsibilities, and limited documentation. A modular design separated the calibration system from the main game logic, allowing independent development.
8. **Amadeo device availability:** Limited access to the actual Amadeo device required development and testing primarily in PC (keyboard) mode. Full testing with the actual device and real patients remains necessary.
9. **Lack of patient data:** No existing data was available on parameter effects. The CSV files `Trial_5_runs.csv` and `Trial_Random_Parameters.csv` were manually created by testing different parameter combinations to identify main factors influencing difficulty.

7 Conclusion and Future Work

This project extended the Aqua Guardian rehabilitation game by developing an automated difficulty calibration framework that replaces manual parameter tuning with a structured, trial-based workflow. The system collects patient-specific performance data during brief calibration sessions and uses it to train a personalized regression model with adaptive regularization on engineered gameplay features, estimating appropriate difficulty configurations for each patient. A hierarchical optimization strategy then generates a table of calibrated difficulty levels, corresponding to target remaining oxygen values from 10% to 90% in 10% increments. Beyond the calibration pipeline, the project also improved overall system robustness and usability by resolving software bugs, enhancing the user interface, and refactoring the codebase into clearer, modular components.

The system reduces therapist workload and improves session consistency by automating parameter selection, eliminating the need for manual trial-and-error configuration that previously required restarting sessions when parameters proved suboptimal. The multi-target optimization approach provides therapists with multiple calibrated difficulty levels, enabling them to select appropriate challenge levels based on clinical context and patient needs.

Future work will focus on evaluating the system with patients at the Loewenstein Rehabilitation Medical Center and iteratively refining it based on clinical feedback. Additionally, cloud-based data storage will be integrated for systematic data collection. Finally, a formal clinical study will assess the system’s usability and effectiveness.

8 References

- [1] Zook, A., Fruchter, E., & Riedl, M. O. (2019). Automatic Playtesting for Game Parameter Tuning via Active Learning. *arXiv preprint arXiv:1908.01417*. Available at: <https://arxiv.org/abs/1908.01417>
- [2] Dynamic game difficulty balancing. Available at: https://en.wikipedia.org/wiki/Dynamic_game_difficulty_balancing
- [3] Ridge regression. Available at: https://en.wikipedia.org/wiki/Ridge_regression
- [4] Kaggle Intro to Machine Learning. Available at: Kaggle
- [5] Scikit-learn Ridge Regression (Cholesky solver). Available at: scikit-learn
- [6] Google ML Crash Course: Ridge Regularization. Available at: Google ML Crash Course
- [7] PyTorch Optimization Tutorial. Available at: PyTorch
- [8] Unity ML-Agents Toolkit. Available at: Unity ML-Agents